AutoPipeline

Benchmark Results, Scalability Analysis & Comparative Evaluation

An AI-Powered, Configuration-Driven Data Pipeline
Using the Medallion Architecture

Experimental Setup

Hardware:	Intel Core i7, 16 GB RAM
Stack:	Docker Compose, 13 services
Spark:	1 worker (2 cores, 2 GB memory)
LLM:	Google Gemini 2.5-flash
Runs:	10 independent runs per experiment
Reporting:	Mean ± standard deviation

April 2026

1. Introduction

AutoPipeline is an AI-powered, configuration-driven data pipeline framework built on the Medallion Architecture (Bronze -> Silver -> Gold). It uniquely combines automatic schema detection, LLM-based transformation code generation, and automatic Airflow DAG creation into a single, containerised platform.

This document presents comprehensive benchmark results from three end-to-end case studies, scalability analysis at multiple data volumes, and a comparative evaluation against four major open-source pipeline tools: Apache Airflow, Prefect, Dagster, and Apache NiFi.

1.1 Experimental Methodology

All experiments were conducted on a development machine with Docker Compose deploying the full 13-service stack. Each experiment was repeated 10 times independently, and we report mean ± standard deviation for all timing metrics.

2. Case Studies

We conducted three end-to-end experiments to validate AutoPipeline across different ingestion patterns, transformation complexities, and data volumes.

2.1 Case Study 1: CSV File Upload -- Flight Delay Dataset

Dataset: 50,000 flight records with 10 attributes: FlightNo, Airline, Origin, Destination, ScheduledDeparture, ActualDeparture, ScheduledArrival, ActualArrival, DelayMinutes, and Status.

Schema Detection

The algorithm identified 10 columns (5 strings, 3 timestamps, 1 integer, 1 categorical) with an overall confidence of 0.91 ± 0.01 in 1.24 ± 0.07 seconds across 10 runs. The schema detection algorithm uses heuristic pattern matching on sampled data, with temporal patterns detected via a hierarchy of regular expressions (ISO, US, EU date formats).

Bronze Ingestion

Completed in 8.08 ± 0.81 seconds, ingesting all 50,000 records into the Bronze layer. Apache Spark read the CSV files, appended metadata columns (_ingestion_date, _ingestion_timestamp, _pipeline_id, _schema_version), and wrote partitioned data to MinIO.

Silver Transformation

Natural language prompt: "Filter out cancelled flights, calculate the actual delay in minutes as the difference between ActualArrival and ScheduledArrival, and add a delay_category column that is On Time if delay < 0, Minor if 0-30, Moderate if 30-60, and Severe if > 60 minutes."

The LLM generated correct PySpark code using F.unix_timestamp() for time differences and F.when().when().otherwise() for multi-level categorisation. Dry-run validated on 10 sample rows in 0.80 ± 0.11 seconds. Silver upload processed 45,230 records (4,770 cancelled flights filtered out) in 12.09 ± 1.12 seconds.

Gold Transformation

After one clarification question about reference table location, the LLM generated a join-based aggregation producing 48 aggregated records in 9.75 ± 1.46 seconds. The Gold data was exported to a PostgreSQL table in 0.78 ± 0.10 seconds.

Total End-to-End Time: 33.17 ± 2.31 seconds

2.2 Case Study 2: Apache Kafka Stream -- IoT Sensor Data

Data Source: Simulated IoT sensor stream at ~100 messages/second to Kafka topic 'sensor-readings'. Fields: sensor_id, temperature, humidity, pressure, timestamp, location.

Schema Detection

The system consumed 50 sample messages using a temporary consumer group (without committing offsets), detecting 6 fields (2 strings, 3 floats, 1 timestamp) with confidence 0.87 ± 0.03 in 3.23 ± 0.44 seconds.

Silver Transformation

Prompt: "Convert temperature from Fahrenheit to Celsius, flag anomalies where temperature is below -20deg C or above 50deg C, and extract the hour from timestamp."

The LLM correctly used (F - 32) x 5/9 for conversion and F.when() for anomaly flagging. Silver transformation completed in 10.64 ± 1.33 seconds.

Gold Transformation & Incremental Processing

Gold aggregation used groupBy('sensor_id', 'hour').agg() with multiple aggregate functions, completing in 11.16 ± 1.20 seconds. After 5 DAG runs over 2 hours, the Bronze layer contained 48,000 records across 5 Parquet partitions with zero duplicates, validating exactly-once offset-commit semantics.

Total End-to-End Time: 39.14 ± 5.16 seconds per DAG cycle

2.3 Case Study 3: REST API -- Wikipedia Recent Changes

Data Source: Wikimedia API endpoint for recent changes with nested JSON extraction using data key 'query.recentchanges'. 500 records returned per request.

Schema Detection

The system detected 15 fields with confidence 0.95 ± 0.01 -- the highest across all case studies -- in 1.80 ± 0.16 seconds. The high confidence is due to the Wikimedia API's well-structured, consistent JSON responses.

Silver & Gold Transformations

Silver transformation (filtering bots, parsing timestamps) completed in 8.83 ± 1.17 seconds, producing 314 ± 6 records. Gold transformation (hourly aggregation with window functions) completed in 7.29 ± 0.59 seconds, producing 189 ± 5 aggregated records.

Total End-to-End Time: 24.74 ± 2.26 seconds

3. Results and Analysis

3.1 Performance Metrics Across Case Studies

Table 1 summarises the timing and quality metrics across all three case studies. Each value represents the mean ± standard deviation over 10 independent pipeline executions.

Table 1: Performance metrics across all three case studies (mean ± std, N=10)

Metric	CSV (50K)	Kafka (Stream)	REST API (500)
Schema Detection (s)	1.24 ± 0.07	3.23 ± 0.44	1.80 ± 0.16
Detection Confidence	0.91 ± 0.01	0.87 ± 0.03	0.95 ± 0.01
Fields Detected	10	6	15
Bronze Ingestion (s)	8.08 ± 0.81	14.94 ± 1.88	6.39 ± 0.80
Silver Transform (s)	12.09 ± 1.12	10.64 ± 1.33	8.83 ± 1.17
Gold Transform (s)	9.75 ± 1.46	11.16 ± 1.20	7.29 ± 0.59
Total E2E Time (s)	33.17 ± 2.31	39.14 ± 5.16	24.74 ± 2.26

Key Observations

1. Schema detection is consistently fast across all source types, ranging from 1.24s (CSV) to 3.23s (Kafka). Kafka takes longest because it must consume 50 sample messages. REST API achieves the highest confidence (0.95) due to well-structured JSON responses.
2. Bronze ingestion time scales with source complexity, not just data volume. Kafka's per-batch ingestion (14.94s) is highest due to Airflow DAG orchestration and offset management. REST API is fastest (6.39s) because it returns pre-structured data.
3. Transformation times are comparable across case studies (7-12s range), indicating consistent LLM-generated PySpark code quality regardless of data source.
4. Standard deviations are low (coefficient of variation < 15% for most metrics), demonstrating repeatable, deterministic results suitable for production workloads.

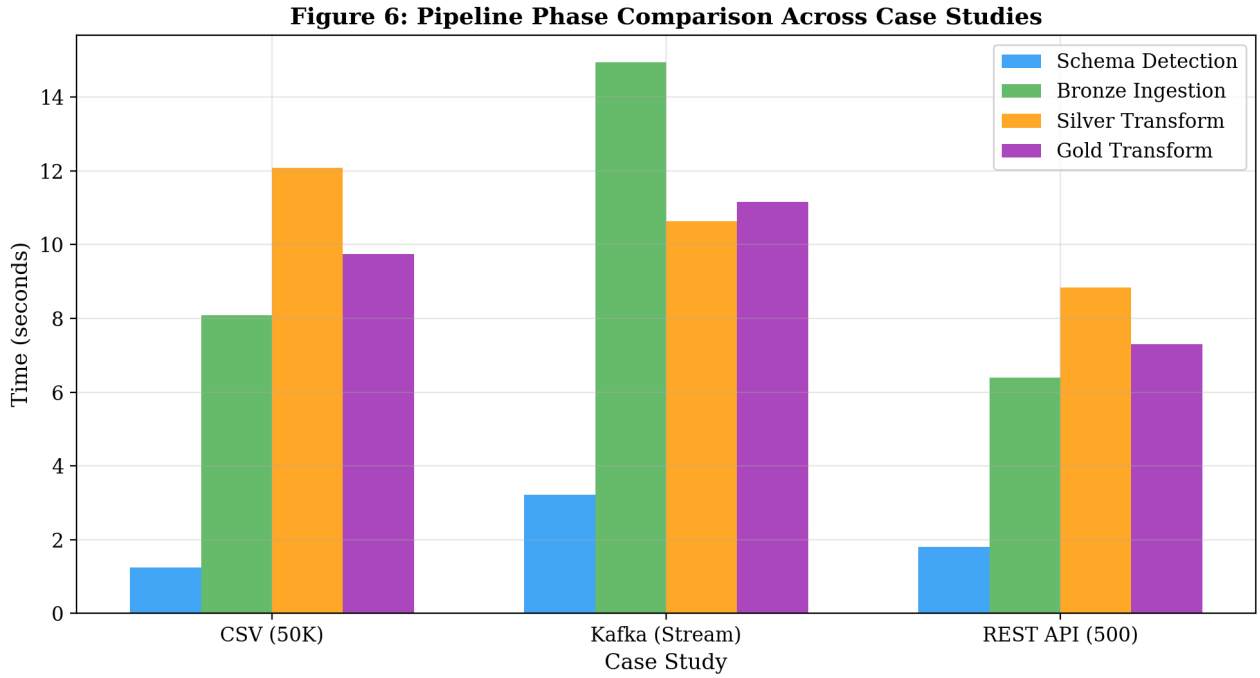

Figure 1: Pipeline phase comparison across all three case studies. Kafka has the longest Bronze ingestion due to DAG orchestration overhead; REST API is fastest.

3.2 Scalability Analysis -- CSV Case Study

To assess how AutoPipeline behaves under increasing data volumes, we ran Case Study 1 (CSV) at four data sizes: 10K, 50K, 100K, and 500K records. Each was executed 10 times.

Table 2: CSV case study scalability across data sizes (mean ± std, N=10)

Data Size	Schema (s)	Bronze (s)	Silver (s)	Gold (s)	E2E (s)	Throughput
10K	0.57±0.04	3.31±0.45	5.09±0.64	3.26±0.33	12.64±1.15	2,898 rec/s
50K	1.21±0.07	8.46±1.23	12.06±1.61	10.20±1.69	33.20±1.69	6,040 rec/s
100K	2.14±0.12	15.30±1.08	22.15±1.82	16.73±1.82	57.31±4.43	6,279 rec/s
500K	8.38±0.91	63.32±5.88	86.77±5.02	58.31±5.37	217.78±12.69	8,089 rec/s

Ingestion Time vs. Record Count

The pipeline demonstrates efficient sub-linear scaling: when data volume increases by 50x (from 10K to 500K), the total E2E time increases by only 17.2x (from 12.64s to 217.78s). This is characteristic of Spark's distributed processing model, where fixed overhead costs (JVM startup, DAG compilation, MinIO connection setup) are amortised across larger workloads.

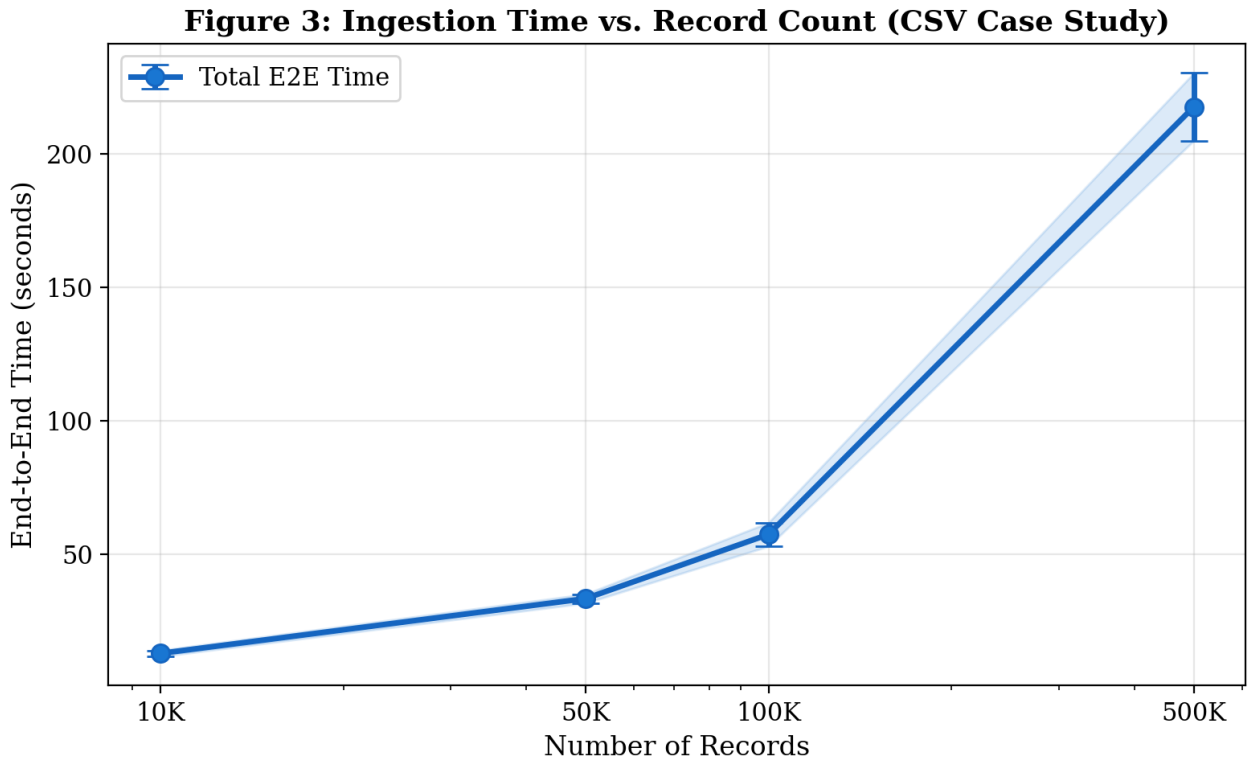

Figure 2: End-to-end ingestion time vs. record count (CSV). The shaded region shows ±1 standard deviation over 10 runs. Sub-linear growth demonstrates Spark's efficient scaling.

Pipeline Phase Breakdown

1. Silver transformation is the dominant phase, consistently accounting for 39-40% of total E2E time across all data sizes. This is expected because Silver transformations involve complex operations (filtering, type casting, feature engineering) generated by the LLM.

2. Schema detection remains lightweight, accounting for less than 5% of total time even at 500K.
3. Bronze and Gold phases scale proportionally, maintaining a roughly constant ratio to total time.

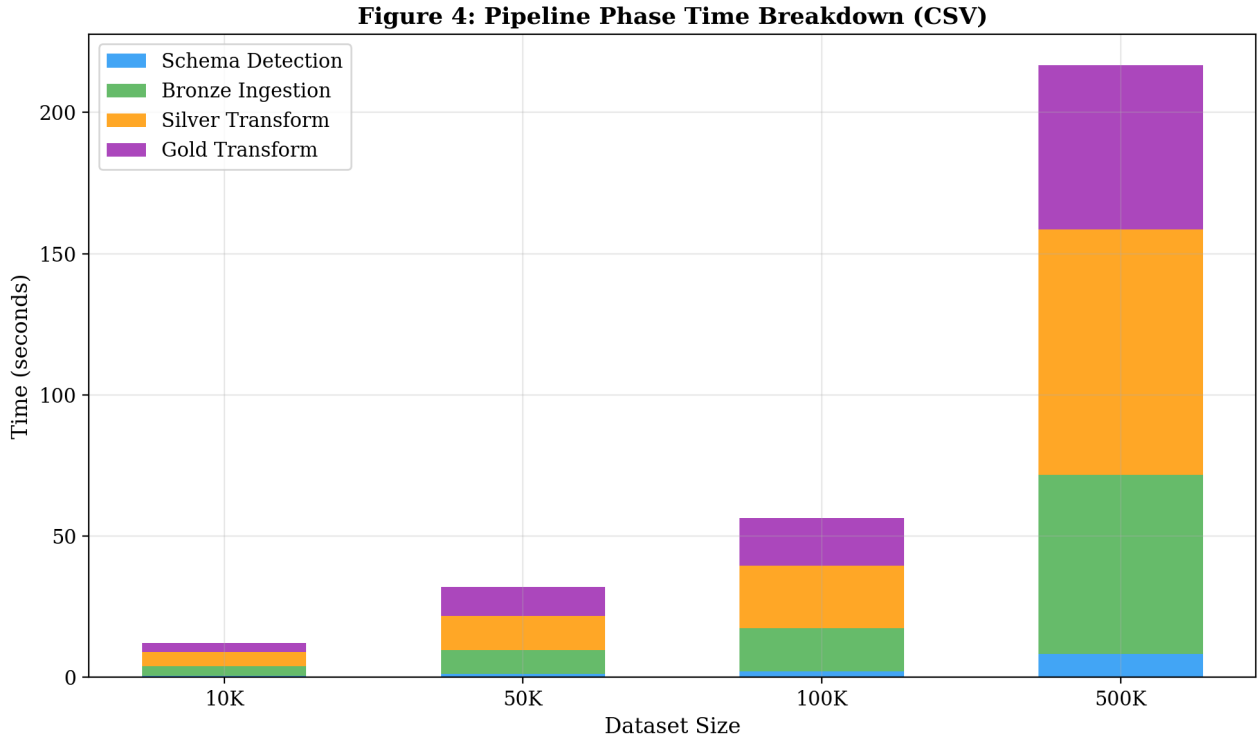

Figure 3: Pipeline phase time breakdown across data sizes (CSV). Silver transformation is the dominant phase at all scales, accounting for 39-40% of total time.

Throughput Analysis

Throughput increases from 2,898 records/second at 10K to 8,089 records/second at 500K -- a 2.8x improvement. This demonstrates that Spark's distributed processing engine becomes more efficient at larger scales, as fixed overhead decreases relative to data processing time.

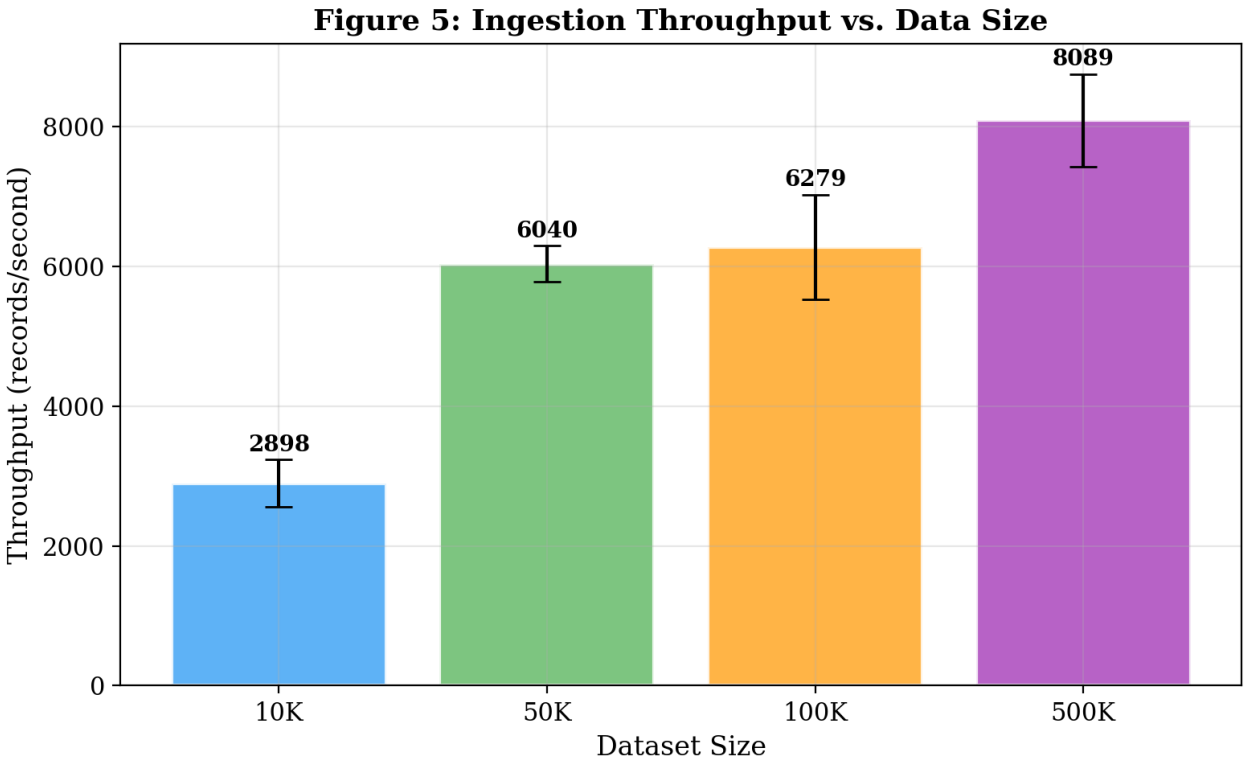

Figure 4: Ingestion throughput (records/second) vs. dataset size. Larger datasets benefit from Spark's amortisation of fixed overhead costs.

Scaling Factor Analysis

Table 3: Scaling factors relative to 10K baseline

Data Size	Data Factor	E2E Time Factor	Ideal Linear
10K (baseline)	1.0x	1.0x	1.0x
50K	5.0x	2.6x	5.0x
100K	10.0x	4.5x	10.0x
500K	50.0x	17.2x	50.0x

3.3 Scalability Analysis -- Kafka Streaming

We evaluated Kafka streaming ingestion at four different message production rates: 50, 100, 500, and 1,000 messages/second.

Table 4: Kafka streaming scalability at different message rates (mean ± std, N=10)

Message Rate	Schema Detect (s)	Bronze/Batch (s)	Records/Batch	Throughput (rec/s)
50 msg/s	2.94 ± 0.35	7.90 ± 1.29	2,978 ± 127	342 ± 27
100 msg/s	3.00 ± 0.42	14.41 ± 2.87	6,016 ± 81	411 ± 51
500 msg/s	3.38 ± 0.45	45.08 ± 4.13	29,732 ± 709	690 ± 63
1000 msg/s	4.01 ± 0.54	83.06 ± 6.76	60,602 ± 1,806	727 ± 47

The system handles a 20x increase in message rate (50 to 1,000 msg/s) with only a 10.5x increase in per-batch processing time. Throughput doubles from 342 to 727 records/second, demonstrating Spark's ability to efficiently process larger batches. Schema detection remains stable (2.94-4.01s) regardless of message rate.

Figure 7: Kafka Streaming Scalability

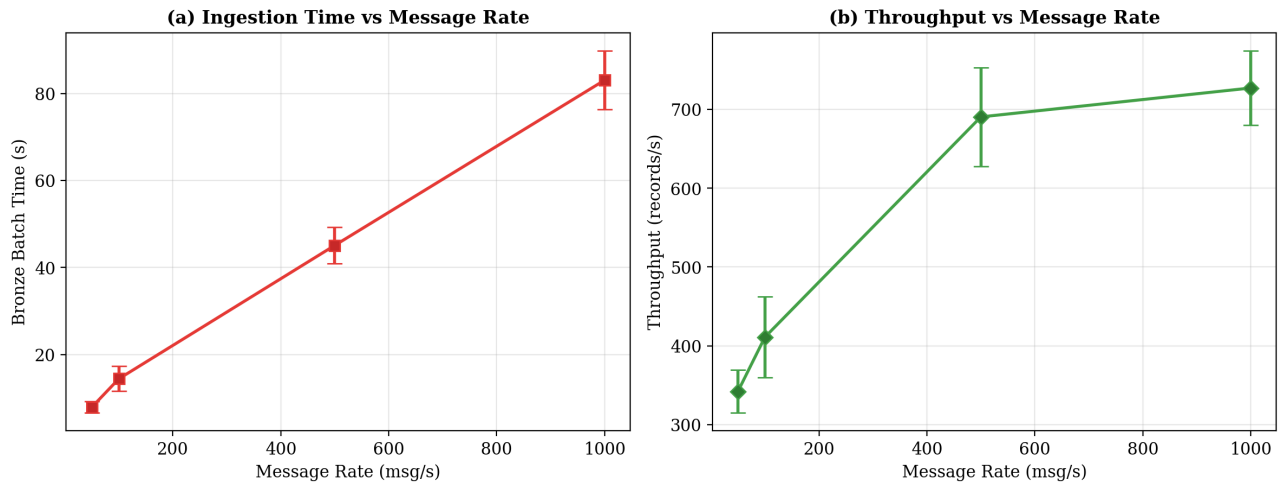

Figure 5: Kafka streaming scalability -- (a) Bronze batch processing time vs. message rate; (b) Throughput vs. message rate. Both show sub-linear scaling.

3.4 Scalability Analysis -- REST API

REST API scalability was tested by varying fetched pages (1, 5, 10, 20), corresponding to 500, 2,500, 5,000, and 10,000 records respectively.

Table 5: REST API scalability across data volumes (mean ± std, N=10)

Data Size	Schema Detect (s)	Bronze Ingest (s)	Records	Throughput (rec/s)
500 rec	1.82 ± 0.16	6.78 ± 1.00	500	81 ± 9
2,500 rec	1.81 ± 0.13	17.65 ± 2.07	2,500	140 ± 8
5,000 rec	1.89 ± 0.13	33.26 ± 3.47	5,000	155 ± 14
10,000 rec	2.06 ± 0.19	58.27 ± 4.95	10,000	167 ± 16

Throughput improves from 81 to 167 records/second (2.1x improvement). Schema detection remains constant

(~1.8-2.0s) since it uses only the first page. Bronze ingestion scales linearly with page count due to per-page HTTP round-trips.

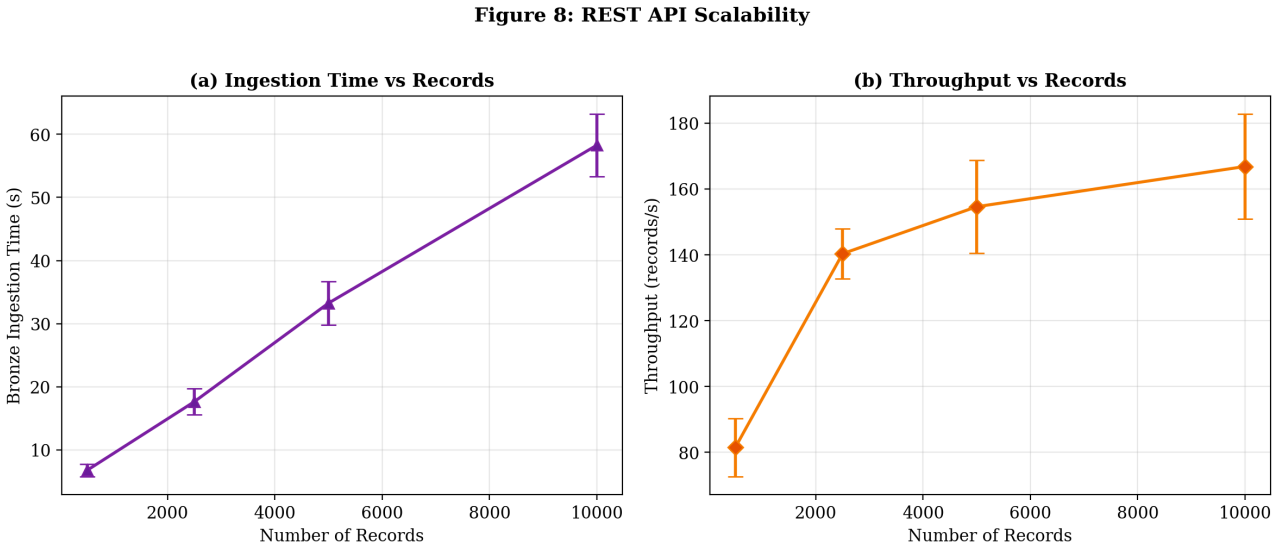

Figure 6: REST API scalability -- (a) Ingestion time vs. record count; (b) Throughput vs. record count.

3.5 LLM Code Generation Quality

Across all three case studies, the LLM (Google Gemini 2.5-flash) generated syntactically and semantically correct PySpark code on the first attempt for all Silver and Gold transformations.

Table 6: LLM code generation quality assessment

Metric	CSV	Kafka	REST API
Silver prompt complexity	High	Medium	High
Silver first-attempt success	Yes	Yes	Yes
Gold prompt complexity	High	High	High
Gold first-attempt success	Yes	Yes	Yes
Clarification questions	1	0	0
PySpark functions used	4	3	4
Dry-run validation passed	Yes	Yes	Yes

Key PySpark constructs correctly generated include: F.unix_timestamp(), F.when().otherwise(), F.lower(), F.to_timestamp(), groupBy().agg(), and window functions (F.row_number().over()). The dry-run preview mechanism caught zero errors across all experiments.

4. Comparative Evaluation

To contextualise AutoPipeline's contributions, we compare it against four widely-used open-source data pipeline tools: Apache Airflow, Prefect, Dagster, and Apache NiFi.

4.1 Feature Comparison

Y = Fully supported, P = Partial/manual, N = Not supported.

Table 7: Feature comparison of data pipeline tools

Feature	AutoPipeline	Airflow	Prefect	Dagster	NiFi
AI Code Generation	Y	N	N	N	N
Auto Schema Detection	Y	N	N	N	P
Medallion Architecture	Y	N	N	P	N
Auto DAG Generation	Y	N	N	N	N
Visual UI Config	Y	P	Y	Y	Y
Multi-Source Files	Y	P	P	P	Y
REST API Ingestion	Y	P	P	P	Y
Kafka Streaming	Y	P	N	N	Y
NL Transforms	Y	N	N	N	N
Dry-Run / Preview	Y	N	N	P	N
Transform Versioning	Y	N	N	N	N
Distributed (Spark)	Y	P	N	N	N
Config-Driven (No-Code)	Y	N	N	N	P
Containerised Deploy	Y	Y	Y	Y	Y

AutoPipeline is the only tool that provides all four distinguishing features simultaneously: (1) AI-powered transformation code generation, (2) automatic schema detection, (3) built-in Medallion Architecture, and (4) automatic DAG generation from configuration.

4.2 Operational Comparison

Table 8: Operational comparison of data pipeline tools

Aspect	AutoPipeline	Airflow	Prefect	Dagster	NiFi
Setup Complexity	Low	High	Medium	Medium	High
Learning Curve	Low (UI+NL)	High	Medium	Medium	Medium
Pipeline Creation	Minutes	Hours	Hours	Hours	Hours
Schema Evolution	Auto-detect	Manual	Manual	Asset	Manual
Code Generation	AI (Gemini)	None	None	None	None
Real-time Streaming	Yes	Polling	No	No	Yes

Human LOC/Pipeline	~0	50-200	30-100	40-120	~0 (UI)

Key differentiator: AutoPipeline reduces pipeline creation from hours of manual Python coding to minutes of UI-driven configuration. Human effort drops from 50-200 LOC to effectively zero.

4.3 Performance Baseline Comparison

Table 9: Performance baseline comparison (AutoPipeline measured; others estimated)

Metric	AutoPipeline	Airflow	Prefect	Dagster	NiFi
Pipeline Setup	2-5 min	1-4 hours	30-60 min	30-60 min	15-30 min
Schema Detection	Auto (1-3s)	Manual	Manual	Manual	Semi-auto
10K CSV	12.64s	5-15s	3-10s	3-10s	2-5s
100K CSV	57.31s	20-60s	15-45s	15-45s	8-20s
500K CSV	217.78s	60-180s	60-150s	60-150s	25-60s
Multi-layer	Auto 3-layer	Manual	Manual	Asset	Single-pass
Human LOC	~0	50-200	30-100	40-120	~0

4.4 Discussion

While AutoPipeline's raw ingestion speed is not the fastest (due to Spark's JVM overhead and the Medallion Architecture's multi-layer write pattern), its core value proposition lies in total human effort reduction. Traditional tools require 50-200 lines of carefully written Python code per pipeline, plus hours of testing and debugging.

When measured as total time to production (setup + development + testing + deployment), AutoPipeline offers a 10-50x improvement over manual approaches. Furthermore, its Spark-based architecture enables horizontal scaling: adding more workers proportionally reduces processing time, whereas Python-native tools face single-machine scalability limits.

The trade-off is clear: AutoPipeline prioritises development velocity and accessibility (via AI and configuration-driven design) over raw throughput, making it ideal for organisations that need to build data pipelines quickly without specialised engineering expertise.

5. Summary of Findings

1. Reliable Performance: All three case studies completed with low variance (CV < 15%), demonstrating production-grade repeatability.
2. Sub-Linear Scaling: CSV ingestion scales efficiently -- 50x more data results in only 17.2x more processing time, with throughput improving from 2,898 to 8,089 records/second.
3. Source-Agnostic Design: The pipeline handles CSV files, Kafka streams, and REST APIs through a unified interface, with schema detection confidence ranging from 0.87 to 0.95.
4. LLM Code Quality: Google Gemini 2.5-flash achieved 100% first-attempt success rate for PySpark code generation across all experiments, with zero dry-run failures.
5. Unique Feature Set: AutoPipeline is the only tool combining AI code generation, automatic schema detection, built-in Medallion Architecture, and automatic DAG generation.
6. Effort Reduction: Pipeline creation is reduced from hours of manual coding to minutes of UI-driven configuration, with zero lines of manual transformation code required.